

Вставьте код из данного примера в файл **switch.c**:

```
#include <stdlib.h>
#include <stdio.h>
#include <ucontext.h>

int main() {
    int done = 0;           // флаг для одного запуска
    ucontext_t one;         // первый контекст
    ucontext_t two;         // второй контекст

    getcontext(&one);       // получаем текущий контекст и сохраняем в
    "one"
    printf("hello\n");      // выводим сообщение

    if(!done) {
        done = 1;           // чтобы выполнить swapcontext только один раз
        // сохраняем текущий контекст в "two" и переключаемся на "one"
        swapcontext(&two, &one);
    }

    return 0;
}
```

```
vboxuser@Linux:~$ gcc switch.c
vboxuser@Linux:~$ ./a.out
hello
hello
```

Чтобы еще больше усложнить ситуацию, мы можем попробовать скомпилировать программу (без использования `register`) с использованием некоторых возможностей оптимизации процесса компиляции. Попробуйте сделать следующее и посмотрите, что произойдет?

```
$ gcc -O2 switch.c
$ ./a.out
:
:
```

[illegible]

```

int main() {
    char stack1[8 * 1024]; // стек для первого контекста
    char stack2[8 * 1024]; // стек для второго контекста

    // Первый контекст
    getcontext(&cntx_one);
    cntx_one.uc_stack.ss_sp = stack1;
    cntx_one.uc_stack.ss_size = sizeof stack1;
    makecontext(&cntx_one, (void(*) (void)) push, 2, 1, 1);

    // Второй контекст
    getcontext(&cntx_two);
    cntx_two.uc_stack.ss_sp = stack2;
    cntx_two.uc_stack.ss_size = sizeof stack2;
    makecontext(&cntx_two, (void(*) (void)) push, 2, 2, 1);

    running = 1; // начинаем с первого контекста
    swapcontext(&cntx_main, &cntx_one); // переключаемся с main на cntx_one

    return 0;
}

```

Когда вы скомпилируете и запустите программу, вы, возможно, не будете в восторге, поскольку ничего особенного не происходит. Мы запускаем программу, а затем передаем управление первому контексту. Этот контекст, конечно, вызовет `push()` и будет заполнять и освобождать стек перед завершением работы, но второму контексту никогда не дается время на выполнение.

```
vboxuser@Linux:~$ gcc yield.c
```

```
vboxuser@Linux:~$ ./a.out
```

```
1  push
```

```
1  push
```

```
1  push
```

```
1  push
```

```
1  push
```

```
1  push
```

```
1  push
```

```
1  push
```

```
1  push
```

```
1  top
```

```
1  pop
```

```
1  pop
```

```
1  pop
```

```
1  pop
```

```
1  pop
```

```
1  pop
```

```
1  pop
```

```
1  pop
```

```
1  pop
```

```
-
```

Однако все готово к переключению между контекстами. Единственное, что нам нужно сделать, это добавить вызов `yield()` в процедуру `push()`. Давайте вызовем `yield`, когда окажемся на вершине стека.

```
else {  
    printf("%d%s top\n", p, i, " ");  
    yield(); // переключаем контексты на верхнем уровне стека  
}
```

Теперь ситуация начинает выглядеть более интересной: **два контекста по очереди выполняются**. Вы можете вызывать `yield()` на каждом уровне рекурсии, если хотите, и увидеть, как **выполнение переключается с одного контекста на другой**. Прежде чем мы закончим, нам нужно исправить ещё одну вещь.

```
vboxuser@Linux:~$ ./a.out
```

```
1  push
1  push
1  push
1  push
1  push
1  push
1  push
1  push
1  push
1  top
```

```
-yield-
```

```
2  push
2  push
2  push
2  push
2  push
2  push
2  push
2  push
2  push
2  top
```

```
-yield-
```

```
1  pop
```

```
-yield-
```

```
1  pop
1  pop
1  pop
1  pop
1  pop
1  pop
1  pop
1  pop
1  pop
1  pop
```

lock_Peterson.c:

Модифицировать, чтобы размер счетчика передавался в командной строке при запуске.

При каком значении счетчика в цикле алгоритм перестает работать. Почему?

```
vboxuser@Linux:~$ ./a.out 1000  
counter = 2000  
vboxuser@Linux:~$ ./a.out 10000  
counter = 20000  
vboxuser@Linux:~$ ./a.out 100000  
counter = 200000  
vboxuser@Linux:~$ ./a.out 1000000  
counter = 1999897  
vboxuser@Linux:~$
```

Атомарный обмен

lock_sync_tas.c:

Модифицировать, чтобы размер счетчика передавался в командной строке при запуске.

Подсчитать, сколько раз каждый поток пребывал в состоянии busy-waiting.

```
vboxuser@Linux:~$ gcc lock_sync_tas.c
vboxuser@Linux:~$ ./a.out
vboxuser@Linux:~$ ./a.out 100
counter = 200
all in spin = 0
vboxuser@Linux:~$ ./a.out 100000
counter = 200000
all in spin = 2
vboxuser@Linux:~$ ./a.out 1000000
counter = 2000000
all in spin = 13133
vboxuser@Linux:~$ ./a.out 10000000
counter = 20000000
all in spin = 258676
```

lock_atomic_tas.c:

Модифицировать, чтобы размер счетчика передавался в командной строке при запуске.

Подсчитать, сколько раз каждый поток пребывал в состоянии busy-waiting.

```
vboxuser@Linux:~$ gcc lock_at_tas.c
vboxuser@Linux:~$ ./a.out 1000
counter = 2000
all in spin = 0
vboxuser@Linux:~$ ./a.out 10000
counter = 20000
all in spin = 16338
vboxuser@Linux:~$ ./a.out 100000
counter = 200000
all in spin = 157865
```

lock_sync_cas.c:

Модифицировать, чтобы размер счетчика передавался в командной строке при запуске.

Подсчитать, сколько раз каждый поток пребывал в состоянии busy-waiting.

```
vboxuser@Linux:~$ gcc lock_sync_cas.c
vboxuser@Linux:~$ ./a.out 10000
counter = 20000
all in spin = 0
vboxuser@Linux:~$ ./a.out 100000
counter = 200000
all in spin = 0
vboxuser@Linux:~$ ./a.out 1000000
counter = 2000000
all in spin = 3967649
vboxuser@Linux:~$ ./a.out 10000000
counter = 20000000
all in spin = 31073768
```

lock_sync_tas_yld.c :

Модифицировать, чтобы размер счетчика передавался в командной строке при запуске.

Подсчитать, сколько раз каждый поток пребывал в состоянии busy-waiting.

```
vboxuser@Linux:~$ gcc lock_2.c
vboxuser@Linux:~$ ./a.out 1000
counter = 2000
all in spin = 0
vboxuser@Linux:~$ ./a.out 10000
counter = 20000
all in spin = 0
vboxuser@Linux:~$ ./a.out 100000
counter = 200000
all in spin = 1
vboxuser@Linux:~$ ./a.out 1000000
counter = 2000000
all in spin = 26800
```

lock_sync_tas_ftx.c :

Модифицировать, чтобы размер счетчика передавался в командной строке при запуске.

Подсчитать, сколько раз каждый поток пребывал в состоянии busy-waiting.

```
vboxuser@Linux:~$ gcc lock.c
vboxuser@Linux:~$ ./a.out 1000
counter = 2000
all in spin = 0
vboxuser@Linux:~$ ./a.out 10000
counter = 20000
all in spin = 2
vboxuser@Linux:~$ ./a.out 100000
counter = 200000
all in spin = 63
vboxuser@Linux:~$ ./a.out 1000000
counter = 2000000
all in spin = 776
```